@codedsoul_05

DATA STRUCTURES & ALGORITHMS

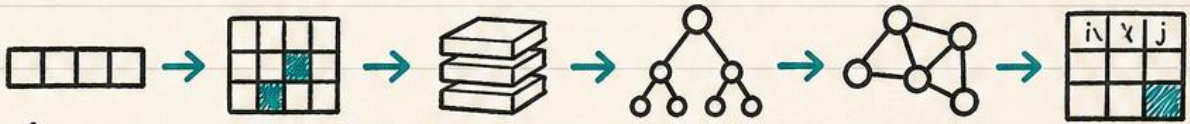
INTERVIEW-READY

— COMPLETE PREMIUM NOTES —

Complexity • Patterns • Practice

MISSION: *Evergreen DSA notes — how to think in code under constraints.*

DSA is patterns + Big-O judgment — recognize, choose, implement, prove.



BIG-O • TWO POINTERS • BFS/DFS • HEAP • DP • GREEDY

WHAT'S INSIDE

- ✓ Interview-focused notes
- ✓ Pattern breakdowns
- ✓ Visual explanations
- ✓ Code templates
- ✓ Common pitfalls
- ✓ Practice problems

BY THE NUMBERS

20 PAGES
high-signal content

PATTERNS
curated & explained

PITFALLS
so you don't repeat them in interviews

TOPICS

STRUCTURES
core building blocks

ALGORITHMS
techniques & strategies

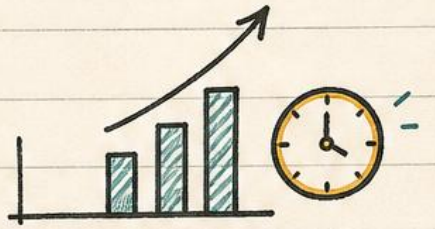
PATTERNS
recognize & apply with confidence

Name the pattern. Justify the complexity.

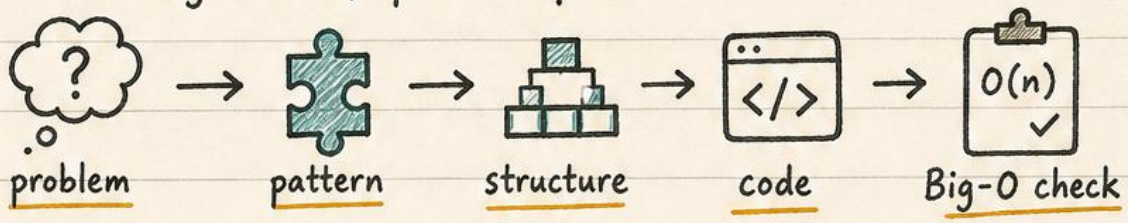
01 — WHAT & WHY DSA

1. DSA = organizing data + transforming it efficiently.

2. Why interviews: predict scale, correctness, tradeoffs under time.



3. Three layers: data structures, algorithms, problem patterns.



4. Goals: correctness first, then time/space, then elegance.



5. Study loop:



6. Mistake: memorizing solutions without recognizing patterns.



TAKEAWAY

Patterns beat memorization.

02 — BIG-O COMPLEXITY

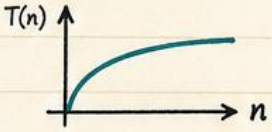
Describes how runtime (or space) grows as input size n grows.

① $O(1)$ — Constant. Doesn't grow with n.



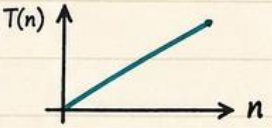
Examples: array access, hash lookup (avg)

② $O(\log n)$ — Logarithmic. Very slow growth.



Examples: binary search, balanced tree ops

③ $O(n)$ — Linear. Grows proportionally.



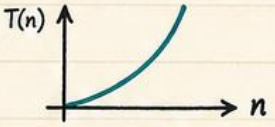
Examples: scan array, find max

④ $O(n \log n)$ — Linearithmic. Common for efficient divide & conquer / sorting.



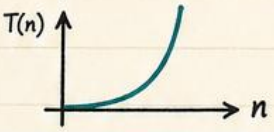
Examples: merge sort, heap sort

⑤ $O(n^2)$ — Quadratic. Nested loops (both ~ n).



Examples: bubble sort, pairwise compare

⑥ $O(2^n)$ — Exponential. Explodes quickly!



Examples: subset enumeration, naïve recursion

SPACE vs TIME TRADEOFF

Using more space (e.g., cache, precompute, hash) can reduce time — and vice versa. Pick based on constraints!

AMORTIZED vs WORST CASE

Amortized: average cost per op over a sequence.

Worst case: maximum cost for any single op.

Know which one you're using.

COMMON PITFALLS

- Nested loops ≠ always n^2. If inner shrinks, total can be $O(n)$, $O(n \log n)$, etc.
- Hash tables: avg $O(1)$, worst case $O(n)$.

COMMON STRUCTURES — COMMON OPS

Structure	Operation	Average	Worst Case
Array	Access by index	$O(1)$	$O(1)$
Hash Table	Lookup / Insert	$O(1)$	$O(n)$
Binary Heap	Push	$O(\log n)$	$O(\log n)$

★ Assumes good hash function & load factor.

INTERVIEW TIP

When it matters, state:

- ✓ Best case
- ✓ Average case (expected)
- ✓ Worst case

Clarify assumptions!

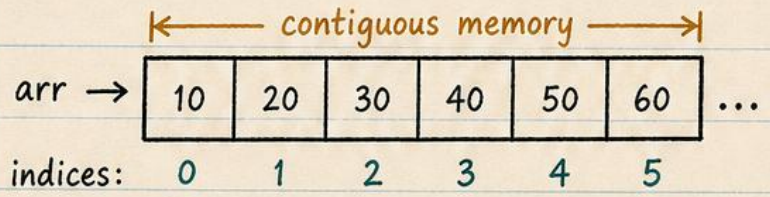
COMMON MISTAKE

Claiming $O(n)$ when you rebuild/redo work every time. Rebuilding a structure per query? That's often $O(n^2)$ overall. Watch for hidden work in loops and helper calls.

TAKEAWAY: Complexity is a promise about growth. ✓

03 – ARRAYS & STRINGS

1. ARRAYS: CONTIGUOUS MEMORY

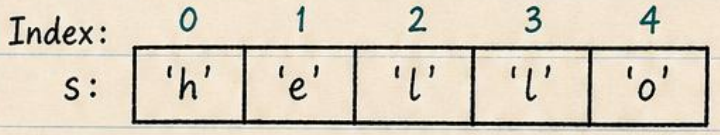


2. OPERATIONS

- Access arr[i] → O(1)
- Insert / Delete in middle → O(n)

3. STRINGS AS ARRAY OF CHARS

String s = "hello"



Strings are just arrays of chars under the hood!

4. COMMON PATTERNS (INTRO)

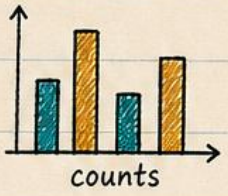
- **Prefix Sums**

prefix[i] = prefix[i-1] + a[i]

ex: a = [2, 3, -1, 4]
prefix = [2, 5, 4, 8]

- **Frequency Counts**

count elements using a hash map / array of size K



5. IN-PLACE vs EXTRA SPACE

In-place: modify input array/string
✓ better space (O(1))

Extra space: use additional arrays, maps, etc.
✓ simpler code sometimes

6. INTERVIEW TIP

Always clarify whether strings are mutable or immutable in your target language!

7. COMMON MISTAKE

Off-by-one errors on slice / subarray bounds.

arr [l ... r]
↑ inclusive ↑ inclusive

Be careful!

TAKEAWAY: Arrays are random-access foundations.

04 — Two Pointers & Sliding Window

1) Two Pointers (Pair Sum)

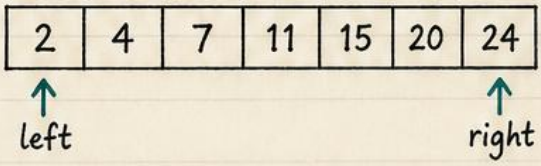
Use left and right pointers on a sorted array to find a pair with given sum = target.

APPROACH

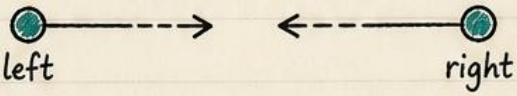
- Sort the array (if not sorted).
- left = 0, right = n-1
- While left < right:
  - sum = a[left] + a[right]
  - If sum == target → found
  - If sum < target → left++
  - If sum > target → right--

VISUAL

Sorted array



Opposite ends converge towards each other.



Sliding Window

Maintain a window [left, right] that satisfies a constraint.

⊕ EXPAND RIGHT
Include a[right] to the window.

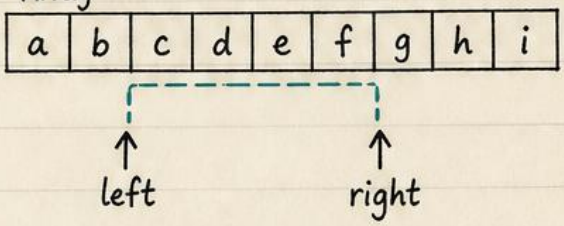
⊖ SHRINK LEFT
While constraint is violated, move left forward.

EXAMPLE CONSTRAINTS

- Max sum ≤ k
- Longest substring with at most K distinct characters

WINDOW VISUAL

Array

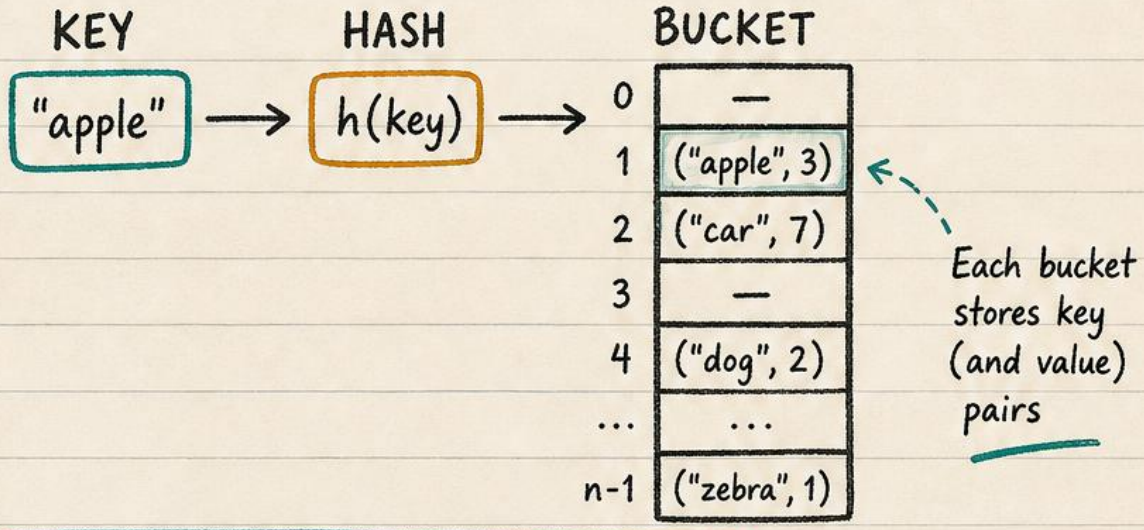


Current window = [left, right]

WINDOW STATE / VARS	left, right, count / sum (or other info)
WHEN TO USE	Contiguous subarray / substring problems.
COMPLEXITY	O(n) after sorting if needed (Two Pointers becomes linear after sort)
COMMON MISTAKE	Moving both pointers incorrectly / Not updating window state.

TAKEAWAY: *One pass with smart indices.*

05 — HASHING & HASH MAPS



☆ Avg O(1) lookup / insert / delete

• USE FOR:

- frequency map
- set membership
- two-sum
- anagrams
- caching indices

• COLLISIONS

Different keys can hash to the same bucket. Handle with chaining (linked list / vector) or open addressing (probing). Good hash function helps distribute keys evenly.

TRADEOFF:

SPACE for TIME

INTERVIEW:

Clarify key type (string/int/obj). Ask about collision expectations and constraints (n, value range).

MISTAKE:

Mutating keys after insertion or relying on order of elements in a hash map.

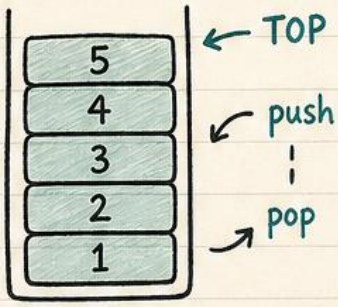
TAKEAWAY:

Hash when you need fast lookup by key.

06 — STACKS & QUEUES

STACK (LIFO)

Last In, First Out



OPERATIONS (All O(1))

- push(x) → add to top
- pop() → remove & return top
- peek()/top() → return top
- isEmpty() → check empty

USE CASES

★ Matching Parentheses ()[]{}
★ Undo / Redo in Editurs
★ Monotonic Stack Problems

MONOTONIC STACK TIP

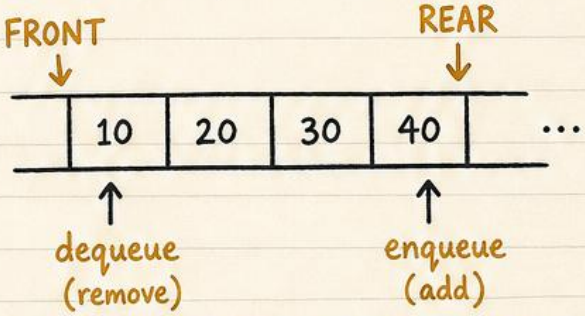
For Next Greater Element (NGE):

1. Traverse from right to left
2. Keep a decreasing stack
3. Pop while stack.top ≤ current
4. NGE is stack.top (if exists)
   Then push current

Example: nums = [2, 1, 2, 4, 3]
NGE = [4, 2, 4, -1, -1]

QUEUE (FIFO)

First In, First Out



OPERATIONS (All O(1))

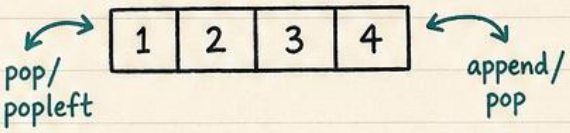
- enqueue(x) → add to rear
- dequeue() → remove & return front
- front()/peek() → return front
- isEmpty() → check empty

USE CASES

★ BFS (Breadth-First Search)
★ Task Scheduling / Fairness
★ Rate Limiting / Buffering

DEQUE (DOUBLE ENDED QUEUE)

Insert / Delete at both ends in O(1)



Use Deque for Sliding Window Maximum
(Maintaining candidates)

COMMON MISTAKE

Using list.pop(0) as Queue in Python.
It is O(n) because it shifts all elements.

```
q = []
q.append(1)   # O(1)
x = q.pop(0)  # O(n) ✗
```

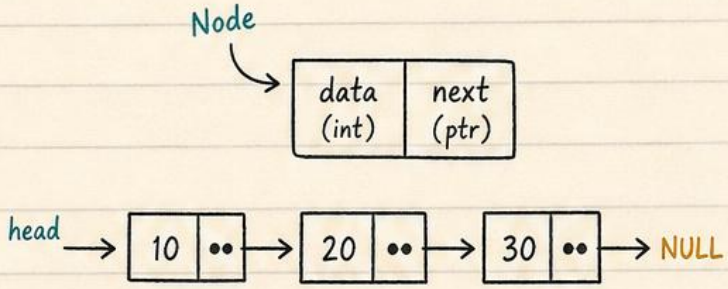
} Avoid!

★ **TAKEAWAY:** Order of removal is the design.

07 — LINKED LISTS

WHAT IS IT?

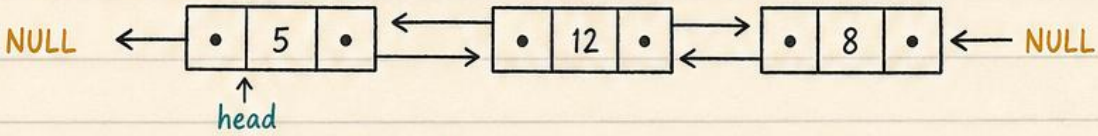
A linear data structure of nodes. Nodes are not stored contiguously in memory. They point to the next node.



- **SINGLY LINKED LIST**



- **DOUBLY LINKED LIST**



COMMON OPERATIONS

- ☑ Insert at head O(1)
- ☑ Delete at head O(1)
- ☑ Search (by value) O(n)

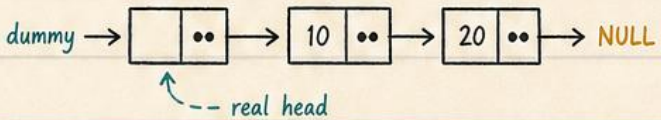
CLASSIC PROBLEMS ☆

1. Reverse a linked list
2. Detect cycle (Floyd's Tortoise & Hare)
3. Merge two sorted linked lists
4. Find middle of list (Slow & Fast Pointers)

TIP: Use a DUMMY HEAD

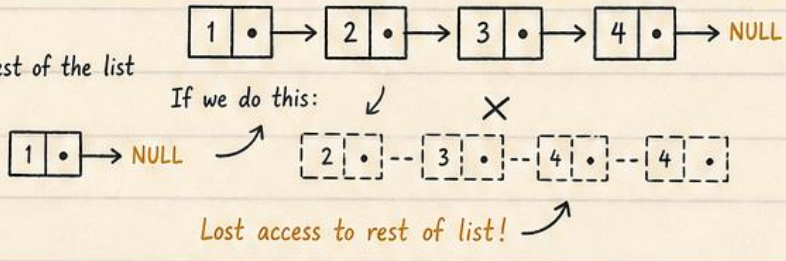
Simplifies edge cases for insertions / deletions.

Example (insert at head)



⚠ COMMON MISTAKE:

Losing the pointer to the rest of the list while changing links.



☆ TAKEAWAY: Pointer care > clever tricks.

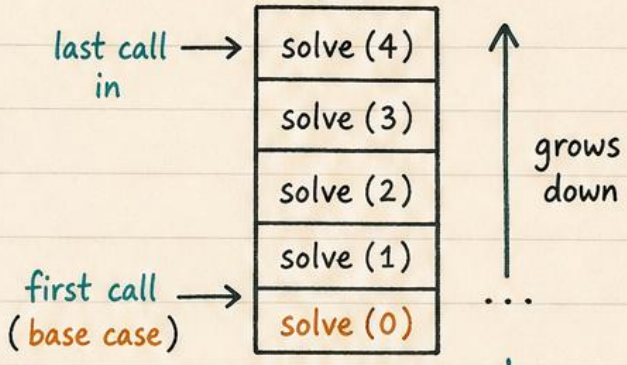
08 – RECURSION & BACKTRACKING

• RECURSION

= base case + recursive case

```
def solve(...):
    if (base case):
        return ...
    # recursive case
    solve(smaller problem)
```

CALL STACK (mental model)



Each call waits for the one below to finish.

• BACKTRACKING TEMPLATE

1. CHOOSE — make a choice / take cation → 2. EXPLORE — recurse to go deeper → 3. UNCHOOSE — undo the choice / restore state

This is the magic that makes back-tracking work!

• CLASSIC PROBLEMS



SUBSETS | PERMUTATIONS | N-QUEENS | COMBINATION SUM

PRUNING — use constraints to cut off useless paths early.

⚠ COMPLEXITY OFTEN EXPONENTIAL — SAY IT ALOUD.

! COMMON MISTAKES

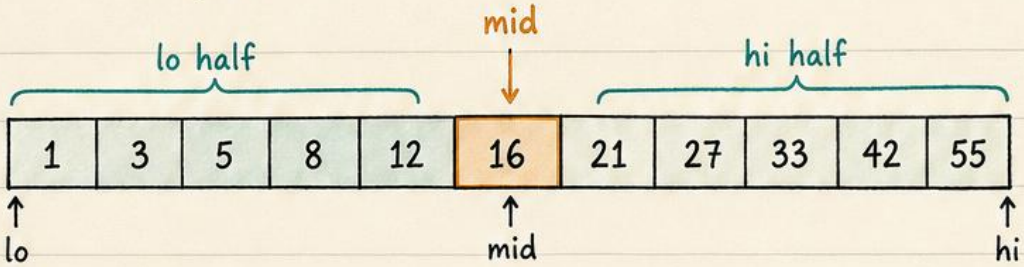
1. Missing base case → infinite recursion ∞
2. Not restoring state (not unchoosing) → corrupts the search

★ **TAKEAWAY:** Explore systematically; undo cleanly.

09 – BINARY SEARCH

① IDEA

Work on sorted data. Repeatedly halve the search space.



② TEMPLATE (search x)

```
lo = 0,  hi = n - 1
while  lo <= hi:
    mid = lo + (hi - lo) // 2
    if  a[mid] == x:
        return mid
    elif  a[mid] < x:
        lo = mid + 1
    else:
        hi = mid - 1
return -1    # not found
```

HOW IT WORKS

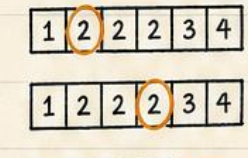
- Compare at mid.
- If smaller → discard left half.
- If larger → discard right half.
- Repeat until lo > hi.

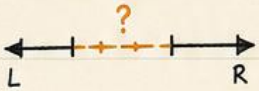
③ COMPLEXITY

TIME: O(log n)
SPACE: O(1)

④ VARIANTS

- ★ First Occurrence (lower bound)
- ★ Last Occurrence (upper bound)
- ★ Search Insert Position (first index where a[i] >= x)
- ★ Answer-Space Search (min capacity / max speed / ...)





binary search on answers using isOk(mid)

⑤ PREDICATE

Define a boolean check:

isOk(mid)

Monotonic:

- False ... False, True ... True
- Lets us binary search on the answer space.

⑥ COMMON MISTAKE

⚠ Infinite loop due to wrong mid bias or off-by-one updates.
Be consistent with boundaries!

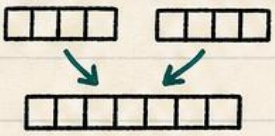
EXAMPLES

- Using mid = (lo + hi) // 2 but doing lo = mid (not mid + 1)
- Using hi = mid (not mid - 1)

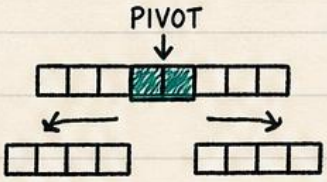
TAKEAWAY: Search on sorted space or monotonic answer.

10 – Sorting Essentials

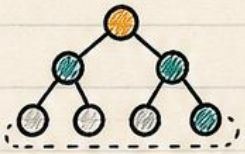
① Comparison Sorts



- **Merge Sort** | $O(n \log n)$ STABLE

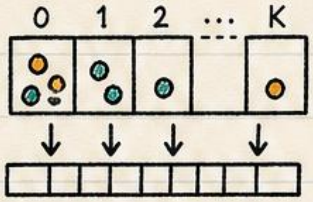


- **Quick Sort** | Avg: $O(n \log n)$, Worst: $O(n^2)$



- **Heap Sort** | $O(n \log n)$ NOT STABLE

② Non-Comparison Sorts

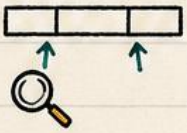


- **Counting / Radix Sort**

Use when keys are in a limited range

> Non-comparison sorts can be linear time!
> e.g., Counting Sort $O(n + k)$

③ ★ When to Sort First?



Enables Two Pointers

Enables Binary Search

④ Stability Matters

For multi-key sorting

(e.g., sort by age, then by name)

⑤ Interview Tip

Know when to use **built-in sort** (e.g., sort() in Python) vs. roll your own (only if required!)

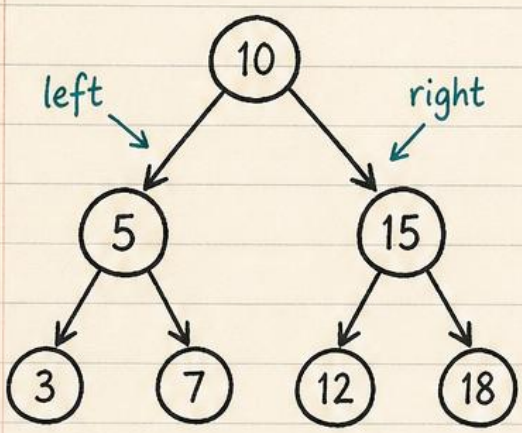
⑥ ⚠ Common Mistake

Sorting when a hash would do the job in $O(n)$



TAKEAWAY: Sort is a gateway pattern.

11 — TREES & BST



TRAVERSALS

- Preorder (Root, Left, Right)
  → [10, 5, 3, 7, 15, 12, 18]
- Inorder (Left, Root, Right)
  → [3, 5, 7, 10, 12, 15, 18]
- Postorder (Left, Right, Root)
  → [3, 7, 5, 12, 18, 15, 10]
- Level-order (BFS)
  → [10, 5, 15, 3, 7, 12, 18]

BST PROPERTY

- For any node:
  `left < root < right`
- Inorder traversal of BST gives sorted order.



HEIGHT

- Height = number of edges on longest path from root to a leaf.

BALANCED TREE

- For every node,
  $|\text{height(left)} - \text{height(right)}| \leq 1$
  Keeps operations ~ $O(\log n)$.

CLASSIC PROBLEMS

- ☐ Lowest Common Ancestor (LCA)
- ☐ Diameter of Binary Tree
- ☐ Validate BST
- ☐ Path Sum (Root to Leaf)

... and many more!

RECURSION

- Recursion is natural on trees.
- Base case: null node
- Process + Recurse on left & right



COMMON MISTAKE

Confusing BST with Heap.
BST → ordered by value (left < root < right)
Heap → ordered by priority (parent ≥ children or ≤ children)

TAKEAWAY:

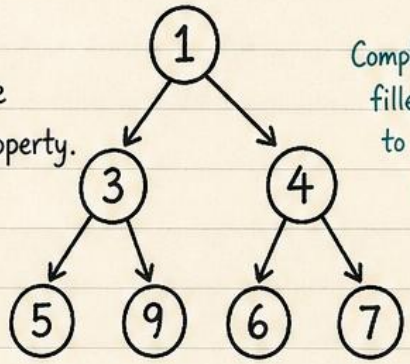
Trees = recursive structure + traversals.

12 – HEAPS & PRIORITY QUEUE

★ HEAP:

A complete binary tree that satisfies the heap property.

For MIN-HEAP: parent ≤ children



Complete tree filled left to right

Array (level order): | 1 | 3 | 4 | 5 | 9 | 6 | 7 |

OPERATIONS

- peek (top) O(1) → root
- push (insert) O(log n)
- pop (extract top) O(log n)

★ USES

- Top-K elements
- Merge K sorted lists
- Dijkstra's shortest path
- Median in a stream (two heaps)

MIN-HEAP vs MAX-HEAP

MIN-HEAP	MAX-HEAP
Parent ≤ Children	Parent ≥ Children
Root = smallest	Root = largest
Use when you need the minimum	Use when you need the maximum

INTERVIEW ANGLE

Q: Why does a heap beat a sorted array for streaming data?

A:
- Heap supports insert & extract in O(log n).
- Sorted array needs O(n) insert (shifting).
- Heap keeps the extreme (min/max) always-ready.

⚠ COMMON MISTAKE

Using a heap when you only need to sort once.

Example:

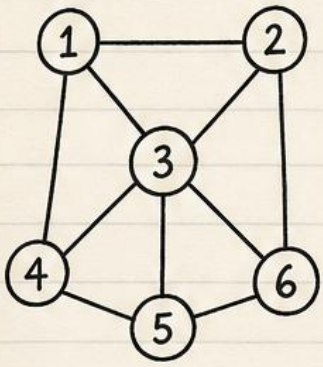
You have all data, sort it once → O(n log n) is simpler & faster than heapify + multiple ops.

Use heap for dynamic/streaming, not for one-shot sorting.

TAKEAWAY: Always-ready extreme value.

13 – GRAPHS – BFS & DFS

1) GRAPH AS ADJACENCY LIST

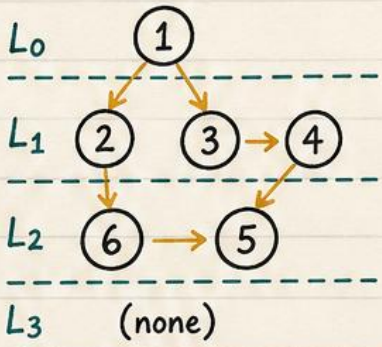


1 : [2, 3, 4]
2 : [1, 3, 6]
3 : [1, 2, 5, 6]
4 : [1, 5]
5 : [3, 4, 6]
6 : [2, 3, 5]

2) BFS (BREADTH-FIRST SEARCH)

QUEUE = SHORTEST UNWEIGHTED PATH / LEVELS

Start at 1



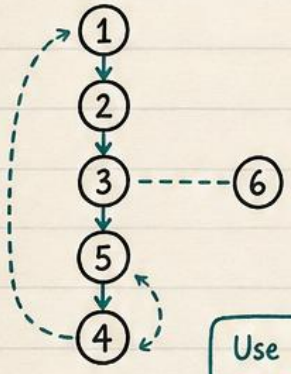
Shortest path to 5 from 1:
1 → 3 → 5
(2 edges)

Use queue.
Process level by level.

3) DFS (DEPTH-FIRST SEARCH)

STACK / RECURSION = REACHABILITY, COMPONENTS, CYCLES

Start at 1



One possible DFS order:
1, 2, 3, 5, 4, 6
(many orders possible)

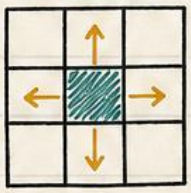
Use stack (iterative) or recursion.

4) VISITED SET IS CRITICAL

Maintain a visited set to avoid revisiting nodes.
Mark when you push/enqueue.



5) GRID AS GRAPH



4-neighbors:



8-neighbors:



Each cell = node. Edges to valid neighbors.

6) COMPLEXITY

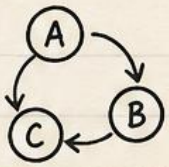
BFS: O(V + E)
DFS: O(V + E)

V = # vertices
E = # edges

7) COMMON MISTAKE



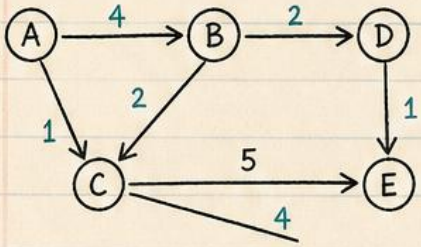
Missing visited set → infinite loops (especially in cyclic graphs or grids)



TAKEAWAY: BFS for distance; DFS for explore/structure.

14 — GRAPHS
PATHS & ORDER

★ DIJKSTRA for non-negative weights (heap + dist[])



idea — Greedy by smallest distance. Use min-heap (node, dist). Relax edges, update dist[].

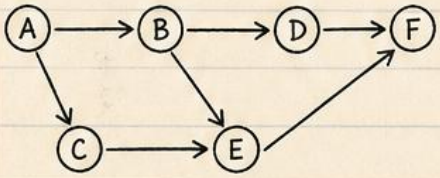
Time: $O((V+E) \log V)$

node	dist[]
A	0
B	4
C	1
D	3
E	4

→ If negative edge weights exist, use BELLMAN-FORD.

BF — Relax all edges $|V|-1$ times. Detect negative cycle on $|V|^{th}$ iteration.

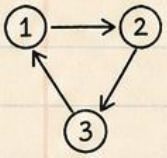
★ TOPOLOGICAL SORT for DAG (Kahn BFS / DFS finish)



- **KAHN (BFS)**
  Use indegree. Push 0-indegree nodes to queue.
  Pop, add to order, reduce indegree.
- **DFS FINISH**
  DFS and push node to stack on finish. Reverse stack.

Order (example): A, B, C, E, D, F ☺

★ DETECT CYCLE

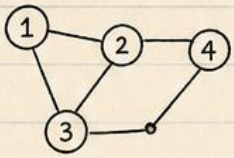


DIRECTED GRAPH

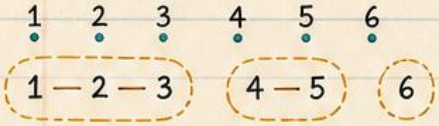
- DFS with recursion stack (3-color: white, gray, black)
- or Kahn's: if topo order size < V, cycle exists.

UNDIRECTED GRAPH

- DFS/BFS with parent tracking. If we visit a neighbor that is not parent → cycle.



★ UNION-FIND for components / MST teaser



Uses:
- Count connected components
- Kruskal's MST uses Union-Find to avoid cycles.

Ops:
find(x)
union(x, y)
+ path compression
+ union by rank

INTERVIEW TIP

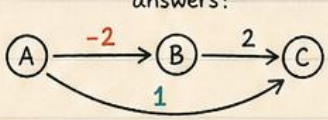
Pick the algorithm based on constraints:

- ✓ Weights non-negative? → Dijkstra
- ✓ Negative weights? → Bellman-Ford
- ✓ DAG order needed? → Topo sort
- ✓ Components / MST? → Union-Find (or Kruskal)

COMMON MISTAKE

Using Dijkstra with negative edges.

✗ Can give wrong answers!



★ TAKEAWAY

MATCH GRAPH PROPERTIES TO ALGORITHM.

15 – Dynamic Programming Intro

DP = overlapping subproblems + optimal substructure.

Two main ways:

Top-down (memoization)
- Solve recursively
- Cache results
- Only compute what's needed

Bottom-up (tabulation)
- Build table iteratively
- Solve small → large
- All states computed in order

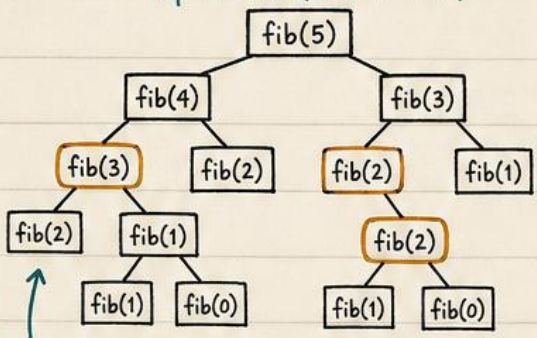
Same idea, different order!

DP Recipe

1. **Define state** → What do we represent?
2. **Transition** → How to go from smaller states?
3. **Base case(s)** → Smallest states / known answers.
4. **Order** → Evaluation order (for tabulation).

Tiny Example: Fibonacci (fib(n))

Top-down (memo tree)



Bottom-up (table)

n	0	1	2	3	4	5
fib[n]	0	1	1	2	3	5

↑ Build left → right

Transition: fib[n] = fib[n-1] + fib[n-2]
Base: fib[0] = 0, fib[1] = 1

Complexity (DP)

Time = (# of states) × (cost per transition)
Space = (# of states) × (space per state)

Count your states. Estimate transition cost. That's your complexity!

When to use what?

DP
- Overlapping subproblems
- Optimal substructure
- Can define states + transitions

Greedy
- Local choice is globally optimal
- Prove with exchange argument / matroid etc.

Plain recursion
- No overlap (rare)
- Or input too small
- Otherwise: exponential!

Common mistake: Unclear or wrong state definition → Wrong recurrence, wrong answer.

What am I really solving?

Takeaway: Name the state. Write the transition. ✓

16 — CLASSIC DP PATTERNS

1 1D CLIMB / HOUSE ROBBER



At i, best answer using first i items (choose or skip).

state: dp[i]

2 KNAPSACK (0/1)



Pick or skip item i within capacity W.

state: dp[i][w]

3 UNBOUNDED KNAPSACK / COIN CHANGE



∞ use items any number of times.

state: dp[i][w]

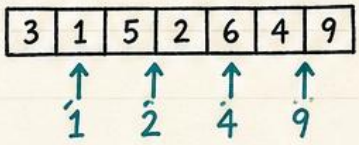
4 LCS / EDIT DISTANCE



Match / skip / replace to align two sequences.

state: dp[i][j]

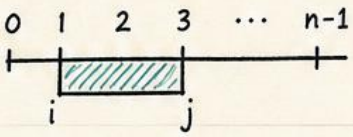
5 LIS (LONGEST INCREASING SUBSEQUENCE)



Best increasing subsequence ending at i.

state: dp[i]

6 INTERVAL DP (TEASER)

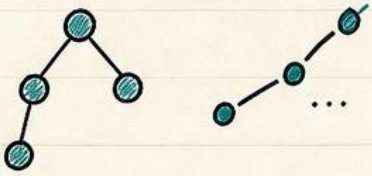


Solve subproblems on intervals [i, j].

state: dp[i][j]

★ Think partition / merge / burst balloons / matrix chain…

7 DP ON TREES / PATHS



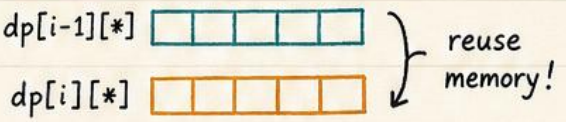
Aggregate from children or along the path.

state: dp[u] or dp[u][k]

SPACE OPTIMIZE (WHEN POSSIBLE)

Use rolling arrays to cut memory:

- 1D DP → keep prev, curr
- 2D DP → keep prev row / column



INTERVIEW TIP

Before coding, identify the PATTERN FAMILY first.

- ✓ Sequence DP
- ✓ Knapsack DP
- ✓ Interval DP
- ✓ Tree DP
- … others

COMMON MISTAKE

Forcing DP on problems that are greedy-solvable.

Always check if a greedy approach works first!

TAKEAWAY: Patterns are reusable templates.

Recognize → Model State → Write Recurrence → Optimize

17 — GREEDY & BIT TRICKS

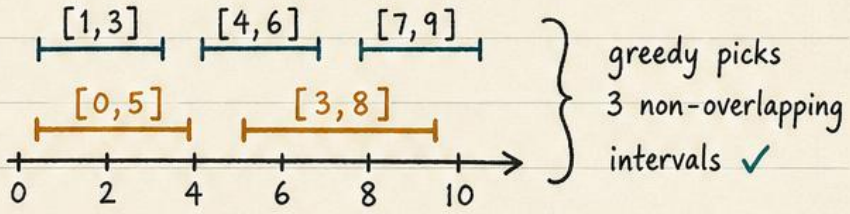
GREEDY

local choice → global optimum
when exchange / greedy property holds.

CLASSICS:

- interval scheduling
- Huffman teaser
- jump game intuition

Interval Scheduling (earliest finish first)



⚠ PROVE OR GIVE COUNTEREXAMPLE.
Greedy choice must be justified!

BIT TRICKS

& AND | OR ^ XOR << LEFT SHIFT >> RIGHT SHIFT

- Check power of 2

```
n & (n-1) == 0   // n > 0
```

8 (1000_2)
1000 & 0111 = 0000 ✓

- XOR for missing / unique

```
a^a = 0 , a^0 = a , a^a^b = b
```

Missing number (1..n)
ans = (1 ^ 2 ^ ... ^ n) ^ (all elements)
1 ^ 2 ^ 3 ^ 4 ^ 5 ^ 3 ^ 1 ^ 2 ^ 5 = 4

- Bitmasks for small n subsets

```
mask from 0 to (1<<n) - 1
i is in set if (mask & (1<<i))
```

n = 3 → 2^3 = 8 subsets

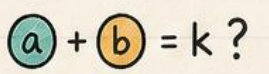
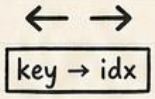
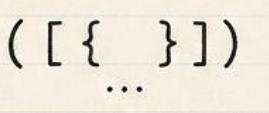
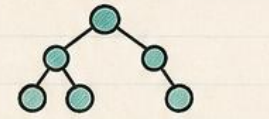
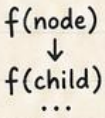
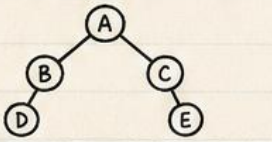
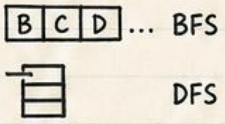
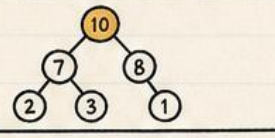
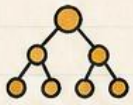
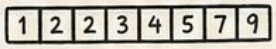
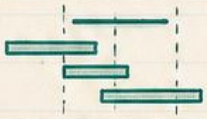
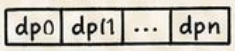
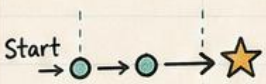
mask (bin)	000	001	010	011	100	101	110	111
subset	{}	{0}	{1}	{0,1}	{2}	{0,2}	{1,2}	{0,1,2}

⊗ **MISTAKE:** assuming greedy works without proof.
Counterexample > wrong solution.

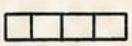
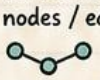
☆ **TAKEAWAY:** Greedy needs a reason; bits compress state.

18 — CHEAT SHEET —

PATTERN MAP

IF YOU SEE X		TRY Y
 Contiguous segment	→	SLIDING WINDOW 
 Pair / sum / exist	→	TWO POINTERS OR HASH 
 Order / nesting	→	STACK 
 Hierarchy	→	TREE RECURSION 
 Relations / paths	→	GRAPH BFS / DFS 
 Extreme / top-K	→	HEAP  max / min heap
 Sorted / monotonic answer	→	BINARY SEARCH 
 Optimal overlapping structure	→	DP define state + transition 
 Local optimal with proof (exchange / greedy choice)	→	GREEDY ✓ choice property ✓ greedy choice ✓ optimal solution

COMPLEXITY STRIP — KNOW O OF COMMON OPS

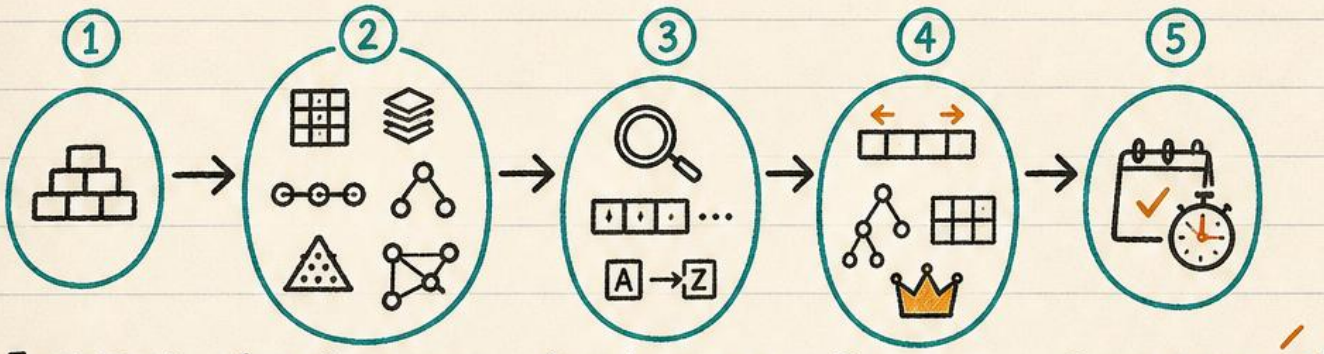
Array access	Push / Pop (stack)	Enqueue / Dequeue	Hash map get / put	Heap push / pop	Sort	Traverse n nodes / edges
 O(1)	 O(1)	 O(1)	 avg O(1)	 O(log n)	 O(n log n)	 O(n + m)

TIP: SAY THE PATTERN NAME OUT LOUD IN INTERVIEW. It shows clarity, structure, and experience.

RECOGNITION IS HALF THE BATTLE.

19 — Roadmap & Recap

You can solve



1. **Foundations** (Big-O, arrays)
2. **Core Structures** (hash, stack, list, tree, heap, graph)
3. **Core Algorithms** (search, sort, BFS/DFS)
4. **Patterns** (window, backtrack, DP, greedy)
5. **Spaced Practice + Timed Mocks**

Recap checklist

- [x] 1. Identify Pattern
- [x] 2. Pick Structure
- [x] 3. Code Carefully
- [x] 4. State Complexity
- [x] 5. Handle Edge Cases

DSA is Reusable Thinking — not a vault of solutions.